

FULL STACK DEVELOPMENT

FASE 2 | AULA 03 -

ROTAS NO EXPRESS.JS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

Nesta aula abordaremos as rotas no Express, um framework amplamente utilizado no desenvolvimento no Node.

Definição

No Express, rotas são endpoints específicos da sua aplicação, correspondendo a URI (Uniform Resource Identifier) e métodos HTTP (GET, POST, PUT, DELETE, etc.).

Criação de rotas

Rotas são definidas usando métodos específicos do Express (app.get, app.post, etc.). Cada rota é associada a um manipulador de rota (função que trata a requisição e gera uma resposta).

Parâmetros de rota

Parâmetros de rota podem ser definidos usando ":" na rota, permitindo capturar valores dinâmicos na URL.

Query Strings

Dados podem ser passados para o servidor através de query strings na URL e acessados no Express através do objeto req.query.

Middleware

Middlewares são funções que têm acesso aos objetos de requisição (req), resposta (res) e à próxima função no ciclo de solicitação-resposta (next). São utilizadas para executar tarefas antes ou depois do manipulador de rota final. Exemplos incluem autenticação, logging e manipulação de erros.

Router

O Router do Express é usado para modularizar e organizar rotas e middlewares, permitindo criar grupos de rotas que podem ser manipuladas separadamente, o que facilita a modularização e organização do código, especialmente em aplicações maiores.

HANDS ON

Nesta seção prática, exemplificaremos todos os tópicos referentes às rotas.



SAIBA MAIS

Métodos HTTP

Falamos sobre os métodos mais utilizados (GET, POST, PUT e DELETE), porém o Express suporta mais alguns.

O método `app.all()` no Express é utilizado para capturar todas as requisições para um determinado caminho. Ele é útil quando você deseja aplicar um determinado middleware ou manipulador de rota para qualquer método HTTP (GET, POST, PUT, DELETE, etc.) em um caminho específico.

```
app.all('/api', (req, res) => {  
  res.send('Esta rota aceita qualquer método HTTP.');
```

```
});
```

Nesse exemplo, qualquer solicitação para o caminho `/api` acionará o manipulador de rota, independentemente do método HTTP utilizado.

O método `app.route()` é utilizado para criar uma instância de roteador para um caminho específico. Ele permite que você defina várias manipulações de rota para esse caminho em uma única instância, melhorando a organização do código.

```
app.route('/livros')  
  .get((req, res) => {  
    res.send('Obter todos os livros');  })  
  .post((req, res) => {  
    res.send('Adicionar um novo livro');  })  
  .put((req, res) => {
```

```
    res.send('Atualizar todos os livros');  
  })  
  .delete((req, res) => {  
    res.send('Deletar todos os livros');  
  });
```

Neste exemplo, a instância de roteador para o caminho /livros é utilizada para definir manipuladores de rota para os métodos HTTP GET, POST, PUT e DELETE, proporcionando uma organização mais clara do código.

O Express suporta todos os métodos HTTP padrão, incluindo aqueles menos comuns, como OPTIONS, TRACE, PATCH, HEAD, etc. Esses métodos costumam ser menos usados diretamente no código do aplicativo, mas são importantes em contextos específicos, como segurança (para o método OPTIONS em CORS) ou otimização de cabeçalhos (para o método HEAD).

OPTIONS

```
app.options('/recursos', (req, res) => {  
  res.sendStatus(200);  
});
```

O método OPTIONS é utilizado para obter informações sobre as opções de comunicação disponíveis para um recurso ou servidor. Pode ser usado para verificar as permissões de CORS (Cross-Origin Resource Sharing) antes de fazer uma solicitação de origem cruzada.

Quando utilizar

- Antes de fazer uma solicitação CORS (usado pelo navegador automaticamente);
- Para obter informações sobre as capacidades do servidor ou recurso.

Vantagens

- Permite que o cliente descubra quais métodos HTTP são permitidos no servidor;
- Facilita a segurança em solicitações de origem cruzada (CORS).

TRACE

```
app.trace('/path, (req, res) => {  
  res.send('Resposta TRACE');  
});
```

O método TRACE é utilizado para realizar um teste de loop de volta no caminho até o servidor. O servidor responde com os cabeçalhos da requisição TRACE no corpo da resposta.

Quando utilizar

- Em cenários de depuração ou diagnóstico para rastrear o percurso de uma requisição.

Vantagens

- Oferece uma maneira de rastrear o trajeto de uma requisição através de intermediários (proxies) até o servidor.

Desvantagens

- Geralmente não é necessário em aplicações convencionais;
- Pode expor informações sensíveis se usado de maneira inadequada.

PATCH

```
app.patch('/recurso/:id', (req, res) => {
```

```
res.send('Recurso parcialmente atualizado');  
});
```

O método PATCH é utilizado para aplicar alterações parciais a um recurso. Ele é comumente usado para atualizar apenas os campos específicos de um recurso, sem modificar os demais. Sua utilização é recomendada ao atualizar parcialmente um recurso, especialmente em operações de atualização parcial.

Vantagens

- Permite a atualização de partes específicas de um recurso sem afetar o restante;
- Reduz a quantidade de dados trafegados na rede.

HEAD

```
app.head('/recurso', (req, res) => {  
  res.send('Resposta HEAD');  
});
```

O método HEAD é idêntico ao GET, mas o servidor não envia o corpo da resposta. É usado para obter apenas os cabeçalhos da resposta e é útil para verificar a existência e status de um recurso sem transferir o corpo completo.

Vantagens

- Reduz o uso de largura de banda, já que apenas os cabeçalhos são transmitidos;
- Útil quando você só precisa das informações de cabeçalho, sem os dados completos.

Desvantagens

- Não pode ser usado para recuperar o conteúdo do recurso;
- Nem todos os servidores ou APIs podem suportar completamente.

Os métodos HTTP menos comuns têm usos específicos, aplicados a cenários particulares. A escolha de utilizá-los dependerá dos requisitos do seu aplicativo e das práticas recomendadas de segurança e eficiência. Em geral, eles oferecem funcionalidades únicas, mas devem ser utilizados com cuidado para evitar riscos de segurança ou desempenho desnecessário.

CORS (Cross-Origin Resource Sharing)

Exemplificamos que o OPTIONS pode ser utilizado em um cenário de CORS, então falaremos mais sobre ele. O CORS é um mecanismo de segurança implementado pelos navegadores da web para controlar as requisições HTTP feitas a recursos de um domínio diferente daquele que serviu a página web. Sem ele, os navegadores impõem a "política de mesma origem" (Same-Origin Policy), que proíbe solicitações de diferentes origens.

Princípios básico: origens (Origins)

Uma "origem" é uma combinação de protocolo (http/https), domínio e porta. Exemplo: http://meusite.com e https://outrosite.com são origens diferentes.

Problema com requisições Cross-Origin

Requisições Simples

- Algumas requisições são consideradas "simples" (simple requests) e, por padrão, são permitidas pelo navegador. Exemplos incluem requisições GET e POST simples.

Requisições Não-Simples

- Requisições que não são consideradas simples precisam ser autorizadas pelo servidor usando cabeçalhos específicos de CORS.

Como CORS Funciona

Preflight Requests

- Antes de fazer uma requisição não-simples, o navegador realiza uma pré-requisição (preflight request) usando o método OPTIONS. Em seguida, o servidor responde indicando se a requisição final será permitida.

Cabeçalhos CORS

- Se o servidor permite a origem, ele responde com cabeçalhos CORS, incluindo Access-Control-Allow-Origin.

Configuração do Servidor

Cabeçalho Access-Control-Allow-Origin

- Indica quais origens estão autorizadas a acessar o recurso;
- Pode ser um valor específico (<https://meusite.com>), "*", ou lista separada por vírgulas.

Cabeçalho Access-Control-Allow-Methods

- Especifica quais métodos HTTP são permitidos.

Cabeçalho Access-Control-Allow-Headers

- Indica quais cabeçalhos HTTP customizados podem ser enviados na requisição.

Cabeçalho Access-Control-Allow-Credentials:

- Indica se a requisição pode incluir credenciais (como cookies ou HTTP authentication).

Vantagens de CORS

Segurança e flexibilidade

- Ajuda a proteger os usuários de possíveis ataques;
- Permite que diferentes domínios colaborem e compartilhem recursos.

Desvantagens de CORS

Complexidade adicional, requisições adicionais e possíveis riscos de segurança

- Pode adicionar complexidade à configuração do servidor;
- Requisições preflight podem aumentar o tráfego da rede;
- Configurações inadequadas podem expor recursos sensíveis.

Em resumo, CORS é crucial para a segurança da web ao permitir ou restringir o acesso a recursos entre diferentes origens. Uma configuração cuidadosa é necessária para garantir uma comunicação segura e eficaz entre origens diferentes.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula sobre as rotas no Express, criamos exemplos abordando diversos cenários de rotas.



REFERÊNCIAS

Express. **Basic routing**. [s.d.]. Disponível em: <<https://expressjs.com/en/starter/basic-routing.html>>. Acesso em: 10 de maio de 2024.

Express. **Routing**. [s.d.]. Disponível em: <<https://expressjs.com/en/guide/routing.html>>. Acesso em: 10 de maio de 2024.

MDN WEB DOCS. **Express tutorial part 4: routes and controllers**. [s.d.]. Disponível em: <https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/routes>. Acesso em: 09 de maio de 2024.

PALAVRAS-CHAVE

Palavras-chave: Route, CORS, Node.JS.

EMSE

POSTECH